

Assignment 2: Solve the Weigelt LP Model Using R

Steve Lambert (slamber6)

September 12, 2026

Problem setup

The decision is how many large, medium, and small units to produce at each of the three plants each day. I used nine decision variables. For example, x_{1L} is the number of large units produced at Plant 1. All variables must be nonnegative.

The objective is to maximize daily profit:

$$\text{Maximize } Z = 420(x_{1L} + x_{2L} + x_{3L}) + 360(x_{1M} + x_{2M} + x_{3M}) + 300(x_{1S} + x_{2S} + x_{3S})$$

The model has three types of limits: production capacity at each plant, storage space at each plant, and forecasted sales for each size. Management also requires all three plants to use the same percentage of their excess capacity.

If T_i is the total production at Plant i , the equal-use requirement is:

$$\frac{T_1}{750} = \frac{T_2}{900} = \frac{T_3}{450}$$

I entered this as two equality constraints: $6T_1 - 5T_2 = 0$ and $3T_1 - 5T_3 = 0$.

Build the model in R

```
# Install once if the package is not already installed:
# install.packages("lpSolveAPI")

library(lpSolveAPI)

# Variable order:
# P1_Large, P1_Medium, P1_Small,
# P2_Large, P2_Medium, P2_Small,
# P3_Large, P3_Medium, P3_Small

weigelt_lp <- make.lp(0, 9)

# Profit for large, medium, and small units at each plant
set.objfn(weigelt_lp, c(420, 360, 300,
                        420, 360, 300,
                        420, 360, 300))
invisible(lp.control(weigelt_lp, sense = "max"))

# Daily production capacity by plant
add.constraint(weigelt_lp, c(1, 1, 1, 0, 0, 0, 0, 0, 0), "<=", 750)
```

```

add.constraint(weigelt_lp, c(0, 0, 0, 1, 1, 1, 0, 0, 0), "<=", 900)
add.constraint(weigelt_lp, c(0, 0, 0, 0, 0, 0, 1, 1, 1), "<=", 450)

# Daily storage space by plant
add.constraint(weigelt_lp, c(20, 15, 12, 0, 0, 0, 0, 0, 0), "<=", 13000)
add.constraint(weigelt_lp, c(0, 0, 0, 20, 15, 12, 0, 0, 0), "<=", 12000)
add.constraint(weigelt_lp, c(0, 0, 0, 0, 0, 0, 20, 15, 12), "<=", 5000)

# Forecasted sales limits for large, medium, and small units
add.constraint(weigelt_lp, c(1, 0, 0, 1, 0, 0, 1, 0, 0), "<=", 900)
add.constraint(weigelt_lp, c(0, 1, 0, 0, 1, 0, 0, 1, 0), "<=", 1200)
add.constraint(weigelt_lp, c(0, 0, 1, 0, 0, 1, 0, 0, 1), "<=", 750)

# Equal percentage of excess capacity used
add.constraint(weigelt_lp, c(6, 6, 6, -5, -5, -5, 0, 0, 0), "=", 0)
add.constraint(weigelt_lp, c(3, 3, 3, 0, 0, 0, -5, -5, -5), "=", 0)

variable_names <- c("P1_Large", "P1_Medium", "P1_Small",
                    "P2_Large", "P2_Medium", "P2_Small",
                    "P3_Large", "P3_Medium", "P3_Small")

constraint_names <- c("Plant_1_Capacity", "Plant_2_Capacity", "Plant_3_Capacity",
                     "Plant_1_Storage", "Plant_2_Storage", "Plant_3_Storage",
                     "Large_Demand", "Medium_Demand", "Small_Demand",
                     "Equal_Use_1_2", "Equal_Use_1_3")

dimnames(weigelt_lp) <- list(constraint_names, variable_names)

# Nonnegativity is the default in lpSolveAPI, but I set it explicitly here.
set.bounds(weigelt_lp, lower = rep(0, 9), columns = 1:9)

weigelt_lp

```

```

## Model name:
##   a linear program with 9 decision variables and 11 constraints

```

Solve the model

```

status <- solve(weigelt_lp)
status

## [1] 0

A status value of 0 means that the solver found an optimal solution.

solution <- get.variables(weigelt_lp)

production <- matrix(solution,
                      nrow = 3,
                      byrow = TRUE,
                      dimnames = list(c("Plant 1", "Plant 2", "Plant 3"),
                                      c("Large", "Medium", "Small")))

knitr::kable(round(production, 3),
              caption = "Recommended daily production")

```

Table 1: Recommended daily production

	Large	Medium	Small
Plant 1	516.667	177.778	0.000
Plant 2	0.000	666.667	166.667
Plant 3	0.000	0.000	416.667

```
daily_profit <- get.objective(weigelt_lp)
daily_profit
```

```
## [1] 696000
```

Check the solution

```
plant_capacity <- c(750, 900, 450)
plant_storage <- c(13000, 12000, 5000)
storage_per_unit <- c(20, 15, 12)
sales_limits <- c(900, 1200, 750)

plant_totals <- rowSums(production)
storage_used <- as.vector(production %*% storage_per_unit)
size_totals <- colSums(production)

plant_check <- data.frame(
  Plant = c("Plant 1", "Plant 2", "Plant 3"),
  Units_Produced = round(plant_totals, 3),
  Capacity = plant_capacity,
  Capacity_Used_Percent = round(100 * plant_totals / plant_capacity, 3),
  Storage_Used = round(storage_used, 3),
  Storage_Available = plant_storage
)

demand_check <- data.frame(
  Size = c("Large", "Medium", "Small"),
  Units_Produced = round(size_totals, 3),
  Sales_Limit = sales_limits,
  Unused_Demand = round(sales_limits - size_totals, 3)
)

knitr::kable(plant_check, caption = "Plant capacity and storage check")
```

Table 2: Plant capacity and storage check

	Plant	Units_Produced	Capacity	Capacity_Used_Percent	Storage_Used	Storage_Available
Plant 1	Plant 1	694.444	750	92.593	13000	13000
Plant 2	Plant 2	833.333	900	92.593	12000	12000
Plant 3	Plant 3	416.667	450	92.593	5000	5000

```
knitr::kable(demand_check, caption = "Sales forecast check")
```

Table 3: Sales forecast check

	Size	Units_Produced	Sales_Limit	Unused_Demand
Large	Large	516.667	900	383.333
Medium	Medium	844.444	1200	355.556
Small	Small	583.333	750	166.667

Interpretation

The maximum daily profit is **\$696,000**. The recommended daily production plan is:

- Plant 1: 516.667 large units and 177.778 medium units.
- Plant 2: 666.667 medium units and 166.667 small units.
- Plant 3: 416.667 small units.

Each plant uses all of its available storage and 92.593% of its excess production capacity. This satisfies management's requirement that the same percentage of capacity be used at all three plants. None of the sales limits are reached, so storage space and the equal-capacity rule are the constraints that determine the final plan.

The model treats production as continuous, which is consistent with a linear programming model. The fractional results can be interpreted as average daily production levels. If each unit had to be a whole number, the problem would need integer decision variables.